OPSPULSE

Production-Grade DevSecOps Platform & Reliability Capstone

- Presenter: Monika Bhardwaj | Session 21 DevOps Final Capstone
- Deployment & Reliability Operations Platform
- Automated CI/CD - Trivy Gates - Terraform AWS EKS - Helm & GitOps - Observability
- https://github.com/Monika-Bhardwaj/OpsPulse-Deployment-Reliability-Operations-Platform

2. Project Overview & Purpose

- Centralized tracking of cloud deployments, incidents, and service health in real time.
- Solves visibility silos across microservices and multi-environment infrastructure.
- Full delivery lifecycle: code to cloud through an automated, secure pipeline.
- Deployment ledger with status transitions (PENDING > RUNNING > SUCCESS / FAILED).
- Severity-ranked incident management with MTTR tracking.

3. Technology Stack

- Frontend: React 18 + Vite, Nginx runtime, responsive CSS dashboard.
- Backend: Python 3.12, FastAPI, SQLAlchemy 2.0, Alembic, Pydantic V2.
- Database: PostgreSQL 16 (psycopg3 pooling); SQLite for isolated tests.
- Containers: multi-stage Dockerfiles, non-root UID 10001, GHCR registry.
- IaC: Terraform (VPC + EKS 1.31, ap-south-1) - K8s: Helm, Ingress, HPA, ArgoCD.
- Observability: Prometheus /metrics + Grafana dashboard - CI: GitHub Actions + Trivy.

4. System Architecture

- Browsers > AWS Load Balancer > Nginx Ingress Controller.
- Path / serves the React frontend (2 replicas, port 8080).
- Paths /api, /health, /metrics route to the FastAPI backend (2 replicas, port 8000).
- Backend persists to PostgreSQL 16 (classroom: in-cluster PVC; prod: Amazon RDS).
- Prometheus scrapes /metrics; Grafana visualizes rate, latency, errors.
- HPA scales 2-6 replicas at 60% CPU; ArgoCD reconciles Git desired state.

5. Testing & CI/CD Pipeline

- 29 backend pytest tests (health, CRUD incl. PUT/PATCH, validation, 404s, pagination, stats, metrics) - all passing.
- 5 frontend smoke tests + production Vite build in the pipeline test job.
- Failed tests block image build, scan, and publish - quality gate enforced.
- Docker Buildx builds both images tagged with the Git commit SHA (traceable).
- Deploy job verifies the release on an ephemeral Kubernetes cluster.

6. DevSecOps: Trivy Security Gates

- Both images scanned with Trivy on every build: --severity HIGH,CRITICAL --exit-code 1.
- Verified clean scan: 0 HIGH/CRITICAL findings (evidence in demo/evidence/).
- Hardening: multi-stage builds, pip/curl removed from backend runtime, apk/apt upgrades.
- Non-root UID 10001, K8s securityContext, resource limits, no committed secrets.
- Pinned action SHAs, least-privilege GITHUB_TOKEN, SHA tags (never mutable latest for deploys).

7. Terraform: AWS VPC + EKS

- Region ap-south-1; terraform fmt + validate pass; remote-state-safe layout.
- VPC 10.20.0.0/16: 2 public subnets (load balancers) + 2 private subnets (nodes).
- EKS 1.31 with managed node group (t3.medium, min 2 / max 4 / desired 2).
- terraform.tfvars.example ships with zero secrets; state files git-ignored.
- Cost estimate ~$70-120/mo for demo period; documented destroy checklist.
- BLOCKED without credentials: apply/destroy + console evidence need AWS keys.

8. Kubernetes, Helm & GitOps

- Helm chart helm/opspulse: lint-clean, renders 11 resources (deployments, services, ingress, HPA, postgres, secret, PVC).
- 2 replicas each; ClusterIP services; ingress routes / to frontend and /api to backend.
- HPA proven live on kind: 2 > 6 replicas under load, scaled back to 2 after.
- ArgoCD Application proven Synced + Healthy; auto-detects Git commits.
- Real fixes from live runs: BACKEND_HOST env (nginx portability), wait-for-db initContainer, HPA ignoreDifferences.

9. Observability: Prometheus + Grafana

- Backend exposes Prometheus metrics at /metrics (requests, latency histogram, status codes).
- Prometheus target verified UP, scraping the backend service every 10s.
- Grafana provisioned with Prometheus datasource + 8-panel reliability dashboard.
- Panel query sum(rate(http_requests_total[5m])) verified returning live req/s data.
- Dashboard JSON committed at monitoring/grafana-dashboard.json (reproducible import).

10. Troubleshooting Exercises

- Scenario A (ImagePullBackOff): invalid image reference, diagnosed via pod events.
- Scenario B (selector mismatch): service with zero endpoints, fixed via label alignment.
- Both executed live in an isolated namespace with captured evidence.
- Bonus real incidents debugged: nginx upstream DNS crash, helm/ArgoCD ingress conflict.
- Runbooks in troubleshooting/ + docs for each symptom, cause, fix, verify cycle.

11. Verification, Limits & Learnings

- Locally verified: app, 34 tests, hardened images, compose stack, Helm, K8s, HPA, metrics, GitOps, CI green, GHCR push.
- Blocked (no AWS keys for this assignment): EKS provisioning + console evidence.
- Key learning: manifests are claims - only live cluster runs prove routing, scaling, and sync.
- Security is a gate, not a badge: Trivy fails the build on HIGH/CRITICAL.
- Submit: repo URL + this deck + demo video recorded per WALKTHROUGH_SCRIPT.md.